SUB-MODULE 7.1 · FINE-TUNING FUNDAMENTALS

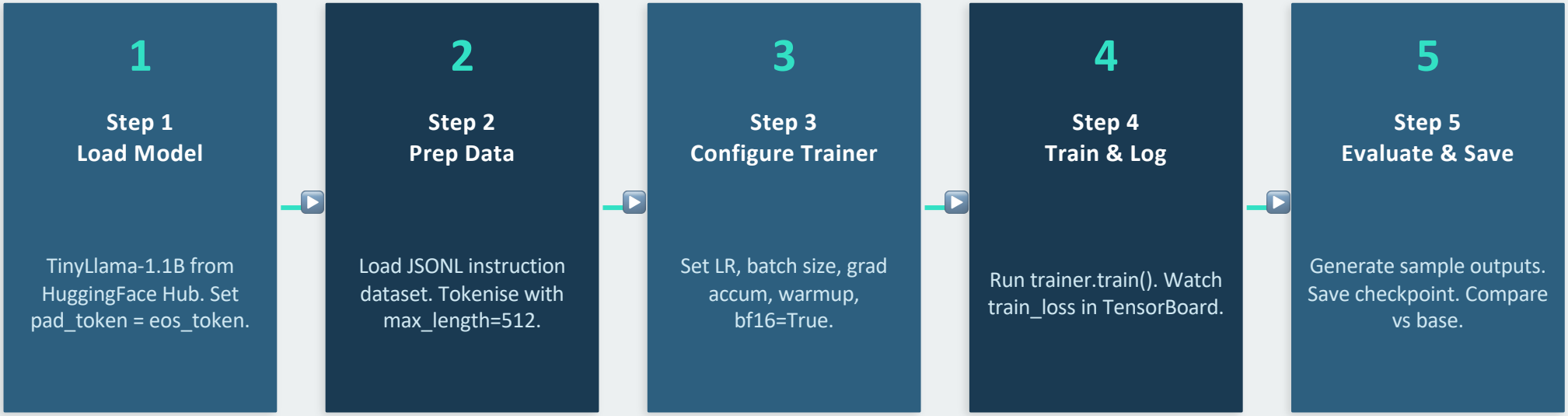
Full Fine-Tuning Implementation

Hands-on: load TinyLlama, fine-tune end-to-end with HuggingFace Trainer, compare loss curves

25 min	B3 – Apply	L3 – Advanced	AI-FT-IN-CD-000001
Duration	Bloom's	Level	ID

Prof. Sashikumaar Ganesan · IISc Bangalore · Advanced Certification in Agentic & Generative AI

Lab 7.1 Roadmap — 5 Steps



Step 1–2: Load Model & Prepare Dataset

```
from transformers import AutoModelForCausalLM, AutoTokenizer
from datasets import load_dataset
import torch

# Load model & tokenizer
model_id = "TinyLlama/TinyLlama-1.1B-Chat-v1.0"
tokenizer = AutoTokenizer.from_pretrained(model_id)
tokenizer.pad_token = tokenizer.eos_token  # required for causal LM

model = AutoModelForCausalLM.from_pretrained(
    model_id, torch_dtype=torch.bfloat16, device_map="auto"
)
print(f"Trainable params: {sum(p.numel() for p in model.parameters()):,}")
# → Trainable params: 1,100,048,640  (all 1.1B)

# Load & tokenize dataset (Alpaca format)
def tokenize(example):
```

All 1.1B parameters are trainable. Note the difference vs LoRA Lab (Lab 7.2) where this number drops to ~20M.

Step 3–4: Configure TrainingArguments & Train

```
from transformers import TrainingArguments, Trainer, DataCollatorForLanguageModeling

training_args = TrainingArguments(
    output_dir="./full_ft_tinyllama",
    num_train_epochs=3,
    per_device_train_batch_size=2,
    gradient_accumulation_steps=16,   # effective batch = 32
    learning_rate=2e-5,
    lr_scheduler_type="cosine",
    warmup_ratio=0.03,
    weight_decay=0.01,
    bf16=True,                         # mixed precision
    logging_steps=20,
    save_strategy="epoch",
    report_to="tensorboard",
)
```

Gradient accumulation simulates a larger effective batch size without more VRAM. bf16 halves memory vs fp32 without quality loss.

Step 5: Save, Load & Evaluate

```
# Save the fine-tuned checkpoint
model.save_pretrained("./full_ft_checkpoint")
tokenizer.save_pretrained("./full_ft_checkpoint")

# Load for inference
from transformers import pipeline
pipe = pipeline("text-generation", model="./full_ft_checkpoint",
                torch_dtype=torch.bfloat16, device_map="auto")

prompt = "### Instruction:\nExplain gradient descent in simple terms.\n### Response:\n"
output = pipe(prompt, max_new_tokens=200, do_sample=False)
print(output[0]["generated_text"])

# Compare: base model (no fine-tuning) vs fine-tuned
base_pipe = pipeline("text-generation", model=model_id,
                     torch_dtype=torch.bfloat16, device_map="auto")
base_out = base_pipe(prompt, max_new_tokens=200, do_sample=False)
```

Key comparison: The base model will produce generic text. The fine-tuned model should follow the ### Response: format and produce instruction-aligned output.

Interpreting Training Metrics — What to Watch

Metric	Good Sign	Warning Sign	Action
Train Loss	Steadily decreasing	Plateaus early / jumps	Adjust LR; check data quality
Eval Loss	Tracks train loss	Diverges upward (overfit)	Reduce epochs; add weight decay
GPU Memory	< 90% utilisation	> 95% (OOM risk)	Reduce batch size; use grad checkpoint
Learning Rate	Warms up then decays	Too high → NaN loss	Halve the learning rate
Gradient Norm	< 1.0 (with clipping)	Exploding > 10	Enable gradient clipping (max_norm=1.0)

KEY REFERENCES

- HuggingFace Transformers. TrainingArguments docs. huggingface.co/docs/transformers
- Loshchilov, I. & Hutter, F. (2019). Decoupled Weight Decay Regularization. ICLR 2019.
- TinyLlama. github.com/jzhang38/TinyLlama

▶ **Next: Concept 7: LoRA — Low-Rank Adaptation (AI-FT-IN-TH-000006)**